

Reporte tecnico

Demo web de chatbot con agente orquestador

Detector de intenciones, refuerzo OpenIA offline y visualizacion de flujo de tokens

Campo	Valor
Proyecto	EjemploApp
Stack	React + Vite + TypeScript / FastAPI + Pydantic
Modalidad	Demo local sin APIs externas
Documento	Reporte de entrega

Indice

1. Resumen ejecutivo
2. Alcance implementado
3. Estructura del proyecto
4. Arquitectura general
5. Backend
6. Modulo OpenIA/OpenAI-compatible
7. Frontend
8. Seguridad implementada
9. Ejemplos de ataques IA incluidos
10. Evidencia de validacion
11. Limitaciones del demo
12. Posibles extensiones
13. Conclusiones

1. Resumen ejecutivo

Se desarrollo un demo web funcional de una aplicacion tipo chatbot con agente orquestador, detector de intenciones, refuerzo OpenIA/OpenAI-compatible en modo offline, reglas de ciberseguridad y visualizacion interna del flujo de tokens.

La aplicacion permite que un usuario escriba un mensaje, lo procese en backend por etapas y visualice en frontend el flujo completo mediante nodos secuenciales, inspector tecnico JSON, metricas de tokens, intencion detectada, agente seleccionado, herramienta ejecutada y reglas de seguridad aplicadas.

El demo corre localmente sin APIs externas, sin claves reales y sin ejecutar codigo arbitrario.

2. Alcance implementado

- Frontend en React + Vite + TypeScript.
- Visualizacion de flujo con React Flow.
- Backend en Python + FastAPI.
- Validacion de requests y respuestas con Pydantic.
- Reglas de ciberseguridad en YAML.
- Pipeline trazable por etapas.
- Detector de intenciones por reglas.
- Modulo de refuerzo de intencion OpenIA/OpenAI-compatible en modo offline.
- Orquestador de agentes.
- Allowlist de herramientas locales.
- Ejemplos educativos de ataques de IA.
- Documentacion tecnica con diagramas y flujos.

3. Estructura del proyecto

```
EjemploApp/  
  backend/  
    app/  
      agents/  
      services/  
      tools/  
      main.py  
      schemas.py  
      security_rules.yaml  
      requirements.txt  
  frontend/  
    src/  
      components/  
      services/  
      styles/  
      types/  
      App.tsx  
      package.json  
      vite.config.ts  
  docs/
```

FUNCIONAMIENTO_Y_FLUJOS.md
REPORTE_TECNICO.md
REPORTE_TECNICO.tex
README.md

4. Arquitectura general

Componente	Responsabilidad
Frontend React	Captura mensajes, renderiza chat, muestra metricas, visualiza nodos y permite inspeccionar etapas.
FastAPI	Expone endpoints HTTP y valida contratos con Pydantic.
Pipeline	Ejecuta las etapas secuenciales y genera trazas estructuradas.
Detector de intencion	Clasifica la solicitud inicial mediante reglas simples.
Refuerzo OpenIA	Ajusta la intencion en modo offline con senales semanticas locales.
Policy guard	Bloquea prompt injection, secretos y exceso de presupuesto.
Orquestador	Selecciona agente y herramienta segun la intencion final.
Agentes	Generan respuestas deterministas locales.
Herramientas	Ejecutan funciones permitidas por allowlist.

5. Backend

El backend expone una API FastAPI con tres responsabilidades principales:

- Recibir mensajes del usuario.
- Ejecutar el pipeline interno.
- Devolver una respuesta estructurada con trazas, seguridad, tokens e intencion.

5.1. Endpoints

Metodo	Endpoint	Funcion
GET	/api/health	Verifica estado del servicio.
GET	/api/security-rules	Devuelve reglas cargadas desde YAML.
GET	/api/ai-attack-examples	Devuelve ejemplos educativos de ataques IA.
POST	/api/chat	Procesa el mensaje y devuelve respuesta, traza, seguridad, tokens, intencion, agente y herramienta.

5.2. Pipeline implementado

- raw_input
- normalization
- tokenization

- entity_extraction
- intent_detection
- intent_reinforcement
- policy_guard
- orchestrator
- agent_selection
- tool_execution
- response_generation
- audit_log

Cada etapa genera los campos stage, status, input_type, output_type, input, output y metadata.

6. Modulo OpenIA/OpenAI-compatible

Se agrego el modulo openia_intent_reinforcer.py. Su funcion es reforzar la intencion detectada usando una capa semantica local. No llama a APIs externas y no requiere OPENAI_API_KEY.

- Si la intencion base es unknown_intent o general_question, puede reemplazarla por una intencion mas especifica cuando encuentra senales suficientes.
- Si la intencion base ya es especifica, puede conservarla y elevar la confianza.
- Si no encuentra senales utiles, conserva la intencion original.
- Si se configura un modo externo no soportado, no realiza llamadas de red y reporta que el modo externo no esta habilitado.

```
Entrada: tokens
Intencion base: unknown_intent
Intencion final: explain_tokens
Refuerzo aplicado: true
Etapas de traza: 12
```

7. Frontend

El frontend esta organizado en tres paneles:

- Panel izquierdo: chat.
- Panel central: flujo React Flow y metricas.
- Panel derecho: inspector tecnico.

La interfaz muestra conversacion, nodos del pipeline, estado de cada etapa, JSON de entrada y salida, conteo estimado de tokens, costo ilustrativo, politica de seguridad, refuerzo OpenIA, agente seleccionado, herramienta ejecutada, reglas activadas y ejemplos defensivos de ataques IA.

8. Seguridad implementada

- Validacion Pydantic para requests.

- Rechazo de mensajes vacios.
- Limite de longitud del mensaje.
- Deteccion simple de prompt injection.
- Deteccion de texto con forma de secreto.
- Redaccion de secretos en trazas.
- Allowlist de herramientas.
- Bloqueo fail closed ante politica insegura.
- Sin ejecucion arbitraria de codigo.
- Sin endpoints destructivos.
- Sin uso de claves reales.
- Sin servicios pagos ni APIs externas.

9. Ejemplos de ataques IA incluidos

Categoría	Ejemplo resumido	Defensa esperada
Prompt injection directa	Ignorar instrucciones anteriores.	Detectar bypass y bloquear.
Extraccion de prompt interno	Pedir prompts o politicas internas.	No revelar configuracion interna.
Abuso de herramientas	Forzar herramientas no autorizadas.	Aplicar allowlist.
Inyeccion indirecta	Documento externo intenta dar instrucciones.	Tratar contenido externo como datos.
Secreto en entrada	Usuario pega token o API key.	Redactar y bloquear segun politica.
Accion destructiva	Saltar confirmacion para eliminar datos.	Requerir confirmacion y permisos.

10. Evidencia de validacion

10.1. Backend

```
python -m compileall -q app
pip check
```

No broken requirements found.

```
/api/health -> OK
/api/ai-attack-examples -> 6 ejemplos
/api/chat con "tokens" -> explain_tokens, refuerzo aplicado, 12 etapas
/api/chat con prompt injection -> bloqueado, 12 etapas
```

10.2. Frontend

```
npm run build
npm audit --audit-level=moderate
```

```
TypeScript compila sin errores.  
Vite genera bundle de produccion.  
0 vulnerabilities.
```

11. Limitaciones del demo

- No usa modelos reales.
- No llama a OpenAI ni a otro proveedor externo.
- El conteo de tokens es aproximado.
- El costo es ilustrativo.
- El detector de intenciones es determinista.
- No persiste auditorias en base de datos.
- No implementa autenticacion ni roles reales.

12. Posibles extensiones

- Integrar OpenAI real mediante una bandera explicita y OPENAI_API_KEY en variable de entorno.
- Validar salidas de modelo con esquemas estrictos.
- Agregar pruebas automatizadas con pytest.
- Persistir auditoria redacted.
- Agregar autenticacion y RBAC.
- Sustituir el estimador por un tokenizer real.
- Exportar trazas como JSON o PDF.
- Agregar vista de comparacion entre intencion base e intencion reforzada.

13. Conclusiones

El demo cumple el objetivo de mostrar de forma educativa lo que ocurre internamente cuando un chatbot procesa un mensaje. La arquitectura separa frontend, backend, reglas, agentes, herramientas, seguridad y trazabilidad.

La aplicacion queda lista para demostraciones locales y como base para futuras integraciones reales, manteniendo por defecto un modo seguro, offline y sin secretos.